# UT2 · Herencia en la ludoteca
## Clases y operaciones de préstamo

**Desarrollo Web en Entorno Servidor · 2.º DAW · Curso 2026/2027**
Departamento de Informática · IES Ramón Arcas Meca · Lorca

# 1. Objetivo y proyecto preparado

Una ludoteca presta juegos de mesa y videojuegos. Crearás una familia de clases que comparta las operaciones de préstamo, renovación y devolución. Utilizarás una clase abstracta, herencia, una interfaz, un trait y excepciones para controlar los errores.

## 1.1 Archivos de trabajo

**1.** Descomprime UT2_Practica_Herencia_Ludoteca_inicio.zip y abre la carpeta ludoteca completa en VS Code.

**2.** Completa M1 a M6 en ese orden. Cada marcador TODO está en el archivo indicado en la tabla. Conserva los nombres y las firmas indicadas.

**3.** Trabaja únicamente en esos seis archivos PHP. public/index.php, src/cargar.php y public/estilo.css están resueltos. El index crea los objetos, ejecuta las operaciones, captura las excepciones y presenta los resultados.

| Tarea | Archivo | Contenido |
|---|---|---|
| M1 | src/Contrato/Prestable.php | Interfaz del préstamo. |
| M2 | src/Excepcion/PrestamoException.php | Excepción propia. |
| M3 | src/Compartido/GeneraEtiqueta.php | Trait para las etiquetas. |
| M4 | src/Modelo/Juego.php | Clase abstracta y operaciones comunes. |
| M5 | src/Modelo/JuegoMesa.php | Especialización de juego de mesa. |
| M6 | src/Modelo/Videojuego.php | Especialización de videojuego. |

Los archivos iniciales contienen la apertura de PHP, declare(strict_types=1) y un comentario TODO. Debes escribir las declaraciones y su implementación. La carga de archivos es manual y ya está facilitada en src/cargar.php.

No hay que programar formularios, entradas de URL, funciones de arrays ni almacenamiento. Los objetos y los días de cada operación están escritos en el index. Cada recarga empieza de nuevo con los mismos datos.

# 2. Contrato excepción y trait

## 2.1 Interfaz Prestable M1

En src/Contrato/Prestable.php, declara la interfaz Prestable con estas firmas públicas. Declara el contrato sin implementar las operaciones.

```
public function prestar(int $dias): void;
public function renovar(int $diasExtra): void;
public function devolver(): void;
```

## 2.2 Excepción de los préstamos M2

En src/Excepcion/PrestamoException.php, crea PrestamoException como clase hija de RuntimeException. Su cuerpo puede quedar vacío: hereda el constructor que recibe el mensaje de error.

Utiliza InvalidArgumentException, que ya existe en PHP, para datos de construcción inválidos y días no positivos. Utiliza PrestamoException cuando los datos son válidos pero el estado del juego o su plazo impiden la operación. El index ya contiene los bloques try y catch.

## 2.3 Etiquetas con el trait M3

**1.** En src/Compartido/GeneraEtiqueta.php, declara el trait GeneraEtiqueta e implementa el método público etiqueta(): string para devolver id + « · » + nombre, consultando id() y nombre() en el objeto que incorpora el trait.

Por ejemplo, un juego con id 101 y nombre Catan devuelve «101 · Catan». El trait construye la etiqueta; los dos métodos de consulta se implementan en Juego.

# 3. Base abstracta Juego

## 3.1 Propiedades y constructor M4

**1.** En src/Modelo/Juego.php, declara Juego como clase abstracta que implementa Prestable e incorpora GeneraEtiqueta con use.

**2.** Declara las cuatro propiedades privadas de la tabla. Los datos comunes se declaran una sola vez en Juego; las clases hijas añadirán sus propios datos.

| Propiedad | Tipo | Valor inicial |
|---|---|---|
| id | int | Recibido en el constructor. |
| nombre | string | Recibido en el constructor sin espacios en los extremos. |
| prestado | bool | false |
| dias | int | 0 |

El constructor público __construct(int $id, string $nombre) rechaza id menor o igual que cero y nombre vacío después de trim. Lanza InvalidArgumentException; si los datos son válidos, guarda id y el nombre recortado con trim.

## 3.2 Consultas plazo y ficha M4

**1.** Añade los métodos públicos id(): int, nombre(): string, prestado(): bool y dias(): int. Cada uno devuelve su propiedad y no la modifica.

**2.** Declara abstracto el método público plazoMaximo(): int, sin cuerpo. Cada clase hija establecerá su propio límite.

**3.** Implementa ficha(): string para concatenar etiqueta(), el estado Prestado o Disponible y los días, con el formato del ejemplo.

```
101 · Catan · Disponible · Días: 0
```

dias representa la duración total concedida para el préstamo actual. Empieza en cero, aumenta al renovar y vuelve a cero al devolver. No se calculan fechas ni días transcurridos.

# 4. Operaciones heredadas

Completa estos tres métodos públicos en src/Modelo/Juego.php, dentro de M4. Las dos clases hijas los heredan. Comprueba las condiciones en el orden indicado y modifica las propiedades solo después de superar todas las validaciones.

## 4.1 Prestar un juego M4

**1.** Implementa prestar(int $dias): void. Si dias es menor o igual que cero, lanza InvalidArgumentException.

**2.** Si el juego ya está prestado, lanza PrestamoException. Si los días superan $this->plazoMaximo(), lanza también PrestamoException.

**3.** Si se admite el préstamo, guarda true en prestado y la duración recibida en dias. El valor exacto del plazo máximo está permitido.

## 4.2 Renovar un préstamo M4

**1.** Implementa renovar(int $diasExtra): void. Si diasExtra es menor o igual que cero, lanza InvalidArgumentException.

**2.** Si el juego no está prestado, lanza PrestamoException. Si dias + diasExtra supera el límite devuelto por plazoMaximo(), lanza también PrestamoException.

**3.** Si se admite la renovación, suma los días extra a dias. prestado sigue siendo true. Por ejemplo, 10 días más 4 son válidos para un JuegoMesa; añadir otro día debe fallar y conservar 14.

## 4.3 Devolver un juego M4

**1.** Implementa devolver(): void. Si el juego no está prestado, lanza PrestamoException.

**2.** En una devolución válida, guarda false en prestado y cero en dias. Después se podrá prestar de nuevo.

Cada excepción lleva un mensaje que explica el rechazo. El tipo de excepción y el estado conservado deben coincidir con las reglas. La redacción del mensaje puede variar respecto a las capturas.

# 5. Especializaciones

## 5.1 JuegoMesa M5

**1.** En src/Modelo/JuegoMesa.php, crea JuegoMesa como hija de Juego y añade una propiedad privada int $jugadores.

**2.** Declara __construct(int $id, string $nombre, int $jugadores). Llama primero a parent::__construct($id, $nombre) para reutilizar las validaciones y la inicialización de la base.

**3.** Después, rechaza jugadores menor o igual que cero con InvalidArgumentException y guarda el valor válido. Añade jugadores(): int como consulta pública.

**4.** Implementa plazoMaximo(): int devolviendo 14. Sobrescribe ficha(): string para devolver parent::ficha() seguido de « · Jugadores: » y el número de jugadores.

```
101 · Catan · Disponible · Días: 0 · Jugadores: 4
```

## 5.2 Videojuego M6

**1.** En src/Modelo/Videojuego.php, crea Videojuego como hija de Juego y añade una propiedad privada string $plataforma.

**2.** Declara __construct(int $id, string $nombre, string $plataforma). Llama primero a parent::__construct($id, $nombre).

**3.** Después, rechaza la plataforma vacía tras trim con InvalidArgumentException y guarda la plataforma recortada. Añade plataforma(): string como consulta pública.

**4.** Implementa plazoMaximo(): int devolviendo 7. Sobrescribe ficha(): string para devolver parent::ficha() seguido de « · Plataforma: » y la plataforma.

```
201 · Mario Kart 8 Deluxe · Disponible · Días: 0 · Plataforma: Nintendo Switch
```

No vuelvas a escribir prestar, renovar ni devolver en estas clases. Juego utiliza $this->plazoMaximo() y PHP ejecuta la versión definida por la clase del objeto. El mismo código heredado aplica así límites diferentes.

parent::__construct(...) ejecuta el constructor de la base sobre el mismo objeto. parent::ficha() recupera la parte común del texto para que cada hija añada su dato sin duplicar esa construcción.

# 6. Comprobar con el index facilitado

## 6.1 Ejecutar la secuencia preparada

**1.** Cuando completes M1 a M6, utiliza el entorno PHP de clase para abrir public/index.php en el navegador. No tienes que modificar este archivo.

**2.** Si utilizas el servidor integrado de PHP, abre la carpeta ludoteca en VS Code y usa Terminal > Nuevo terminal. Desde esa carpeta, escribe la orden siguiente. Abre después la dirección en el navegador y detén el servidor con Ctrl+C al terminar.

```
php -S localhost:8000 -t public
http://localhost:8000/index.php
```

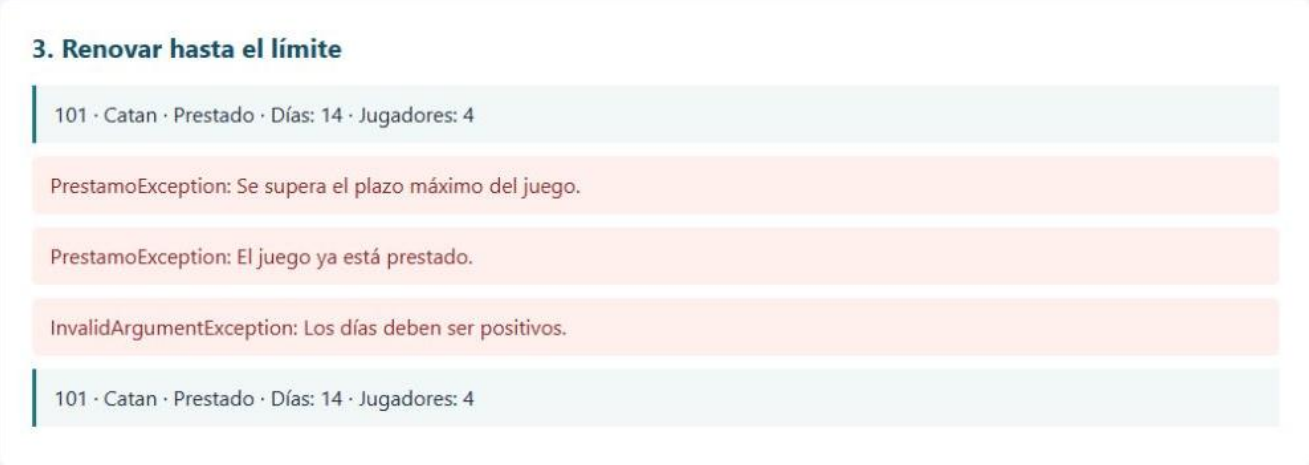
El index crea Catan (101, 4 jugadores), Dixit (102, 6 jugadores) y Mario Kart 8 Deluxe (201, Nintendo Switch). Ejecuta seis bloques: estado inicial; dos referencias al mismo objeto; renovación y rechazos; plazo del videojuego; devoluciones y nuevo préstamo; constructores con datos inválidos.

La asignación $alias = $mesa no crea otro juego. El index presta Catan y consulta el mismo objeto mediante el alias; Dixit conserva su estado independiente. Los bloques siguientes deben continuar después de los errores capturados.

## 6.2 Capturas de comprobación




Inicio de la secuencia: los tres objetos están disponibles y cada ficha incluye el dato de su clase.




Renovación hasta 14 días: las tres operaciones rechazadas conservan el préstamo de Catan en 14 días.